26.0 / 30

In future, please submit your .docx file

SCORE: 0.0/30

Questions

Use the in-class command language syntax as the basis for your command-related answers. ***Do not use the shell on your OS****. Those shells may be inconsistent with each other, behave slightly differently on different machines, use slightly different syntax, and contain unfounded or unadvertised assumptions in their implementations.*

1. Kernel functions are not called by normal procedure calls. The normal procedure call instruction, **`CALL X`**, means to jump to the procedure whose entry point is **`X`**, saving return information to be restored when the procedure returns (the **`RET`** instruction). The kernel call is instruction, **`KC X`**, is restricted to kernel procedures. Whereas normal procedure call lets the caller attempt to jump to any location, kernel call forces entry at the designed kernel entry point. How does this improve security and reliability?

2.5

When working with the kernel, we need to understand that this is one of the most protected subcomponents of the operating system. In addition, calls made here are executed in supervisor mode while normal procedure calls are made at the user level. If an adversary were to gain access to execute commands in supervisor mode, we want to prevent them from accessing/overwriting any location in memory. Forcing kernel calls to enter at a designated location in memory works to prevent damage from adversarial action.

Since system programmers design kernel routines to be reliable, correct, and trustworthy, a common practice in these programs is to begin with validity checks on incoming parameter values. If a parameter fails a test, the program immediately returns an error code without attempting to execute the kernel function.

◦ If a caller could bypass these validity checks—for instance, by tricking the CPU into starting execution after the checks—it could provide invalid data, potentially causing the kernel function to malfunction.

◦ By forcing all kernel calls to start at their designated entry points, the OS prevents such bypass attempts. This ensures that all necessary checks are performed before the core logic of a sensitive kernel function is executed, safeguarding the kernel from malicious or erroneous input.

Guarantees Correct Execution Start: The reliability, correctness, and trust claims of kernel programs assume that any kernel routine is called by always starting execution at its designated first instruction or entry point.

◦ By implementing special mechanisms to ensure that kernel calls always start executing at these designated entry points, the OS ensures a predictable and correct initiation of critical functions. This prevents undefined behavior or crashes that could arise if a kernel routine were to start executing from an arbitrary or incorrect internal point.

2. Why is it safe to execute a kernel function in kernel mode, but unsafe to execute a user function in kernel mode?

2.5

It' s safe to execute a kernel function in kernel mode due to the stringent design principles, validation, and controlled execution environment applied to kernel code. Conversely, it is unsafe to execute a user function in kernel mode because user-level code lacks these critical safeguards and granting it unrestricted system access poses severe risks to system stability and security.

3. Why is some of the kernel, but not all, designated as microkernel?

2.5

Not all of the kernel is designated as a microkernel due to architectural design choices aimed at creating a more modular, reliable, and secure operating system.

In a microkernel architecture:

- Only the most essential and critical functions reside within the microkernel. These typically include:
  - Basic process management: This involves CPU multiplexing (allowing multiple processes to share the CPU by cycling between them in time slices), process creation, and thread management.
  - Main memory management: This ensures separation of address spaces and can include handling data movement between RAM and secondary storage (though some functions might be outside the kernel).
  - Inter-process communication (IPC): This provides messaging services between parallel processes, allowing different components to communicate.
  - Low-level I/O: Basic device management and interrupt dispatching are also part of the microkernel.
- Less critical kernel functions are moved to user space. This includes components like device drivers (beyond the basic low-level I/O), file systems, and network protocols. These are implemented as separate processes or services that run with reduced privileges.

This architectural separation offers several key benefits:

- Improved reliability: If a non-critical service crashes (e.g., a file system or device driver), it does not bring down the entire kernel, as errors in extended functions are "very unusual to crash the kernel".
- Better security: By keeping the microkernel small and in privileged mode, the attack surface is significantly reduced. User processes operate in a non-privileged mode, only switching to kernel mode when explicitly calling a kernel routine.
- Easier maintenance and modularity: Non-critical components can be updated, extended, or replaced independently without requiring modifications to the core kernel. The system is "easier to adapt to different hardware configurations, easier to extend and contract".

The microkernel acts as a core coordinator, handling the fundamental interactions and coordination between these user-space services, which are often communicated using the IPC mechanism.

4. Draw a process pipeline diagram (like those in Chapter 1.2 of Module 1), representing the virtual machines the shell constructs for the typed Unix command **`ls`**, which displays the contents of your current directory. (In Windows the command is **`dir.`**) What info objects are provided as standard **`IN`** and **`OUT`**? Does **`ls`** actually use its standard **`IN`**? Its standard **`OUT`**? How does **`ls`** read the current directory? What process is the parent of **`ls`**?

2.5

- Every process has a standard input port (IN) and standard output port (OUT). For the ls command, the standard IN is connected to the keyboard (an input device). The standard OUT is connected to the display (an output device).
- No, ls does not typically use its standard IN to perform its primary function of listing directory contents. Unlike programs like cat which can read from IN, ls's function is to display information about a directory, not to process data from a stream of input.
- Yes, ls uses its standard OUT. After obtaining the directory contents, ls formats this information and sends it as a stream of bits to its OUT object, which is the display, to show the directory listing to the user.
- The ls program, running as a user process (vmc2), operates in non-privileged user mode. To access a directory, which is a sensitive system resource managed by the OS kernel, ls must make system calls.
    - When a system call is made, the OS automatically places the CPU into privileged kernel mode to ensure safe and controlled access. The kernel routines, designed by expert systems programmers, perform validity checks on incoming parameters before executing the requested function to prevent malfunctions.
- A directory is conceptually a "table, each entry of which associates a symbolic name chosen by the user with an internal pointer to an info object". Directory management functions are implemented within the OS kernel, typically at Level 7 of the OS's hierarchical structure [os99.pdf table 2 (level 7), 81, 82].
- When a user types a command like ls, the shell (vmc1) is the process that creates a new virtual machine (vmc2) for the ls program. Therefore, the shell (vmc1) is the parent process of ls (vmc2). The shell then goes to sleep until its child process (ls) completes its task.

5. Consider the Unix commands

a) **`cat A B C`**

b) **cat A**
c) **cat < A**

2.5 d) **cat < A > D**

Look up the definition of **cat** command and, in each of the above cases, explain what is supposed to happen and draw diagrams showing the process structure the OS sets up. What is the difference between the second and third commands?

[CMD ARGS in-Redir out-Redir]

a. cat A B C

In this case, the shell parses 'cat' as CMD and 'A B C' as ARGS. Cat reads files A, B, and C sequentially and outputs their contents to the display (standard OUT).

b. cat A

In this case, the shell parses 'cat' as CMD and 'A' as ARGS. Cat reads file A and outputs its contents to the display.

c. cat < A

In this case, the shell parses 'cat' as CMD with A as input redirection. File A becomes standard input for cat, which reads from it and outputs to the display.

d. cat < A > D

In this case, the shell parses 'cat' as CMD with A as input redirection. File A becomes standard input for cat, which reads from it and outputs to the display.

6. Consider the Unix command

cat < A | sort

Draw a diagram showing the process structure the OS sets up. A pipe info object has been added. What is a pipe? What output will be generated by this command line? Where will it go?

2.5

a. cat < A | sort

The shell reads the string "cat < A | sort" and parses 'cat' to be the command to be executed. It then parses the string 'A' to be the input redirect, overriding the default input of keyboard. It then parses "|" to pipe the output of the 'cat' command to become the input of the 'sort' command. The 'sort' command will then output to the display (default).

7. Where in a command line do file names appear? What happens if the command line specifies a name for a file that does not exist?

The shell (vmc1), which serves as the command-line interface directly interacting with the kernel, parses each command line into four components: name, arguments (args), input, and output. Filenames can be specified as:

1. As Arguments (args): Many commands accept filenames directly as arguments.

2.0

2. For Input Redirection (<): A filename can be specified as the source for the command's standard input.

3. For Output Redirection (>): A filename can be specified as the destination for the command's standard output.

When a command line specifies a name for a file that doesn't exist, the operation will fail, and the user typically gets an error message. Here's what happens:

How the Directory Manager Handles This (Level 7):

◦ When the shell or a program tries to locate a file by its symbolic name (like text in tbl < text), it makes a SEARCH call to the Directory Manager at Level 7.

◦ The SEARCH operation has a clear rule: it will "Fail if the name does not exist in the given directory" [os99.pdf table 12 (level 7), 265].

◦ Since files must exist within a directory structure to be found, when the SEARCH operation can't find a matching entry for the filename, it means the file simply doesn't exist or isn't accessible through that path.

What This Means for the Shell and Program:

◦ When the SEARCH call fails, the shell (or the program making the call directly, like cat opening its argument files) receives an error code back.

◦ The shell, as the parent process that created the virtual machine for the command, catches this error. Since its job is to "present the response to the user", it displays an error message like "file not found" or something similar.

8. In the command line

```
cat < A | sort
```

3.0

suppose the user inserts "**< B**" immediately after the **cat** command name, intending to read both files **A** and **B** into **cat**:

```
cat < A < B | sort
```

Is this a valid command? Why or why not? If it is not valid, what command would realize the user's intent?

The command cat < A < B | sort is not valid as shells do not allow multiple input redirections - you cannot have both < A and < B in the same command. The second redirection would conflict with the first.
To achieve the intent of reading both files A and B into cat, the correct command would be: "cat A B | sort".

This treats A and B as arguments to cat, which will read both files sequentially and pipe the combined output to sort.

9. Where does the shell find the executable codes for the processes named on the command line?
   - When a user types a command into the shell, the program needs to locate the executable code for that respective command to create and load it into a new virtual machine. The shell finds executable code in the following ways:
   - PATH variable – the vm template includes a PATH component which is a list of directories to be searched for executable file matching command names. This provides the shell with a predefined set of locations where it expects to find executable programs.
   - Directory Manager – To actually find the executable within the specified directories, the shell makes a call to the kernel's directory manager. A directory is a table, managing the information objects (executable files) within the system.
   - SEARCH Operation: The shell performs a SEARCH operation (a Level 7 directory function) on the relevant directories using the command name as the symbolic name. This SEARCH command "locates and returns the handle corresponding to a given external name". For example, h1 := SEARCH(CD, "tbl"); demonstrates the shell searching for the "tbl" program.
   - Once the SEARCH operation returns an internal handle for the executable file, the shell can then instruct Level 8 (Process Management) to load this "named program" (executable code) into the newly created virtual machine for execution.

3.0

10. Does the shell understand the meanings of the commands it executes? For example, would **`cat < out > in`** be a valid command? Diagram the process structure the OS sets up for this command. If it is valid, what does it do?

3.0

As mentioned in class, the shell simply parses through the string it receives and takes the appropriate action given the input and does not possess "intelligence". It does not inherently understand that the input redirects to object 'out' as wrong or an inappropriate method of execution. Since 'out' is assumingly a valid object in this scenario, it takes that as the new input with the object in becoming the new output. If out and in were not valid objects, then we would get an error as the cat vm was being created.